

# TECHNICAL DOCUMENTATION

## RAG-Based Customer Support Assistant using LangGraph & Human-in-the-Loop

### 1. Introduction

#### 1.1 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is an architectural framework that integrates information retrieval systems with large language models (LLMs). Instead of relying solely on parametric knowledge embedded within the model, RAG dynamically retrieves relevant external knowledge from a vector database and uses it as contextual grounding for response generation.

#### 1.2 Need for RAG

Traditional LLM-based systems exhibit the following limitations:

- Lack of domain-specific adaptability.
- Hallucination of factually incorrect responses.
- Static knowledge base with no real-time document awareness.

RAG addresses these challenges by:

- Grounding responses in retrieved contextual data.
- Improving factual accuracy and reliability.
- Enabling domain-specific conversational intelligence without retraining models.

#### 1.3 Use Case Overview

This system is designed as a **Customer Support Assistant** that:

- Processes PDF-based organizational knowledge.
  - Retrieves relevant contextual chunks using semantic similarity search.
  - Generates context-aware responses using LLMs.
  - Routes queries through a structured LangGraph workflow.
  - Escalates uncertain cases to a Human-in-the-Loop (HITL) system.
- 

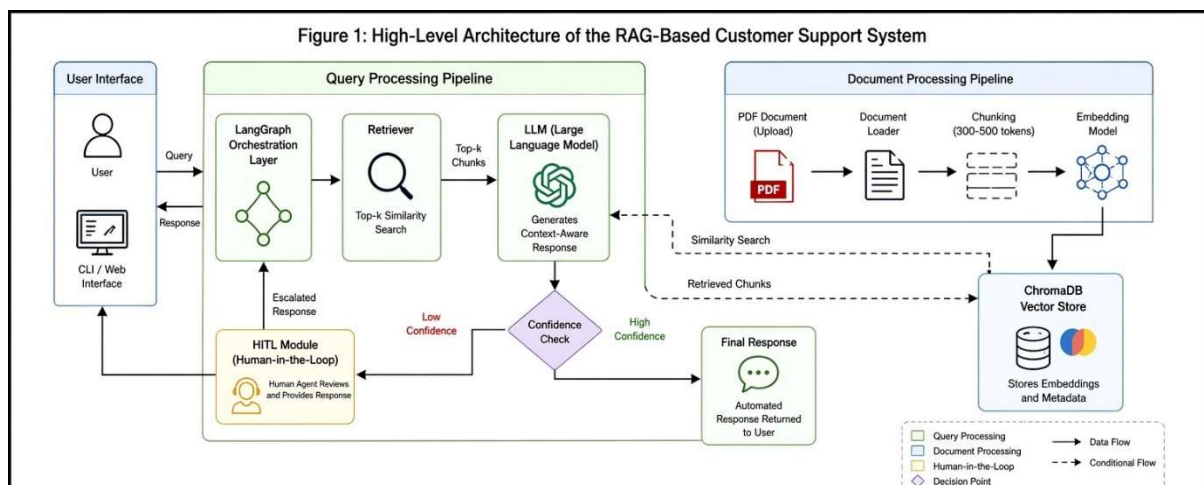
### 2. System Architecture Explanation

The system follows a modular, layered architecture designed for scalability and maintainability.

## 2.1 High-Level Architecture Overview

The architecture consists of the following layers:

- User Interface Layer (CLI / Web Interface)
- Document Ingestion Layer
- Embedding Generation Layer
- Vector Storage Layer (ChromaDB)
- Retrieval Layer
- LLM Processing Layer
- Workflow Orchestration Layer (LangGraph)
- Human-in-the-Loop (HITL) Layer



*Figure 1 – High-Level System Architecture of RAG-based Assistant*

## 2.2 Component Interaction Flow

1. User submits a query via interface.
2. Query is forwarded to LangGraph orchestration layer.
3. Retrieval system fetches relevant document chunks from ChromaDB.
4. Retrieved context is passed to the LLM along with the query.
5. LLM generates a response grounded in retrieved context.
6. Confidence score is evaluated.
7. If confidence is low, the query is escalated to HITL.
8. Final response is returned to the user.

## **3. Design Decisions**

### **3.1 Chunk Size Selection**

Documents are segmented into semantic chunks of 300–500 tokens.

**Rationale:**

- Ensures semantic completeness of each chunk.
- Balances retrieval precision and context availability.
- Prevents excessive noise in vector search results.

### **3.2 Embedding Strategy**

The system uses dense vector embeddings generated via transformer-based embedding models.

**Key characteristics:**

- Semantic similarity-based retrieval.
- Cosine similarity as distance metric.
- Captures contextual meaning beyond keyword matching.

### **3.3 Retrieval Approach**

A Top-K similarity retrieval mechanism is implemented ( $K = 3\text{--}5$ ).

**Justification:**

- Ensures high relevance of retrieved context.
- Reduces computational overhead compared to full document scanning.
- Improves LLM grounding accuracy.

### **3.4 Prompt Design Logic**

The LLM prompt is structured as follows:

- System instruction (behavior definition)
- User query
- Retrieved contextual chunks
- Instruction constraint: answer only from provided context

This design minimizes hallucination and enforces grounded response generation.

---

## 4. Workflow Explanation (LangGraph)

LangGraph is used as the orchestration framework to define a structured and deterministic execution flow.

### 4.1 Node Design

#### Input Node

- Receives and validates user query.
- Initializes graph state.

#### Processing Node

- Performs embedding-based retrieval.
- Passes context to LLM.
- Generates response.

#### Output Node

- Formats and returns final response.

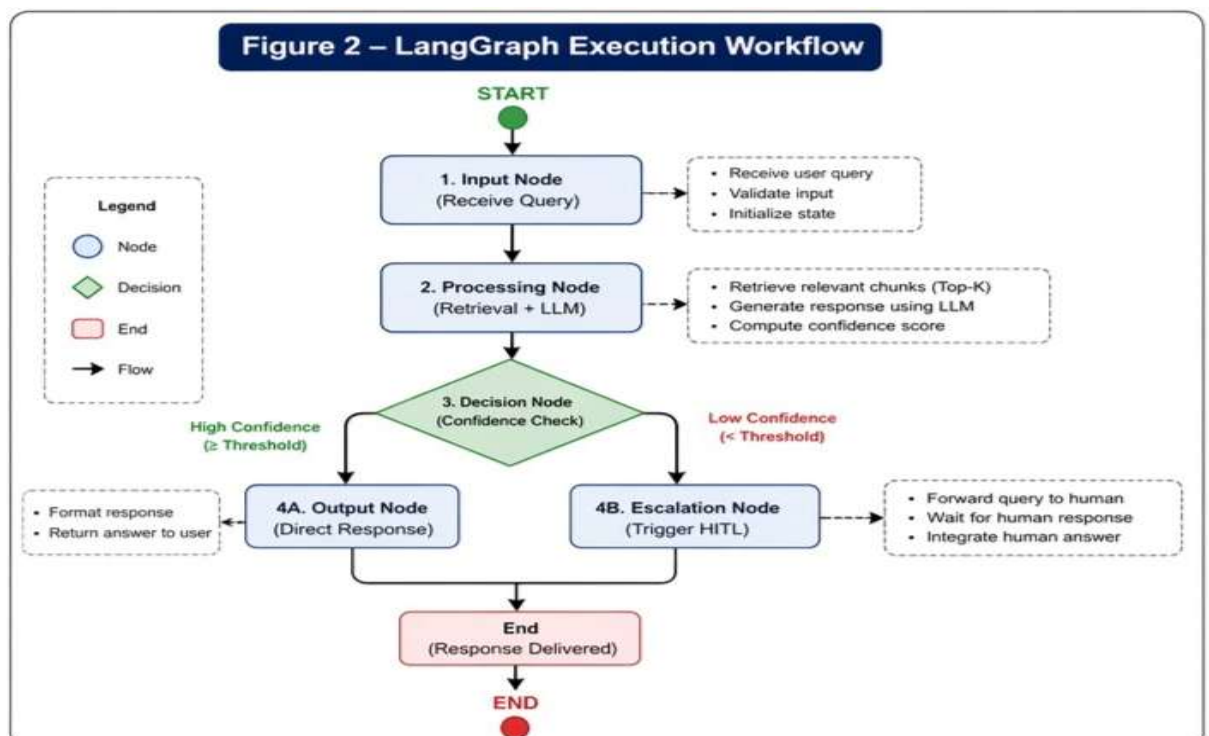


Figure 2 – LangGraph Execution Workflow

### 4.2 State Transitions

A shared state object is passed across nodes containing:

- user\_query
- retrieved\_context
- generated\_response
- confidence\_score
- escalation\_flag

State transitions occur sequentially across graph nodes.

---

## **5. Conditional Logic**

### **5.1 Intent Detection**

User queries are classified into:

- Factual lookup queries
- Procedural queries
- Complex or ambiguous queries

Intent classification is used to determine routing behavior.

### **5.2 Routing Decisions**

Routing is determined based on:

- Retrieval confidence score
- Context relevance threshold
- Query complexity level

### **5.3 Decision Outcomes**

- High confidence → Direct LLM response
  - Low confidence → HITL escalation
  - No retrieval match → fallback or escalation
- 

## **6. HITL (Human-in-the-Loop) Implementation**

### **6.1 Role of Human Intervention**

HITL acts as a validation and correction layer for:

- Low-confidence responses.

- Ambiguous queries.
- Critical or high-impact decisions.

## **6.2 Operational Flow**

1. System flags uncertain query.
2. Query is routed to human operator.
3. Human provides validated response.
4. Response is reintegrated into system output pipeline.

## **6.3 Benefits**

- Improved response reliability.
- Reduced hallucination risk.
- Enhanced trustworthiness in critical domains.

## **6.4 Limitations**

- Increased response latency.
  - Dependency on human availability.
  - Limited scalability under high load.
- 

# **7. Challenges & Trade-offs**

## **7.1 Retrieval Accuracy vs Speed**

Higher retrieval depth improves accuracy but increases latency.

## **7.2 Chunk Size vs Context Quality**

- Smaller chunks: high precision, low context richness.
- Larger chunks: high context, but reduced retrieval accuracy.

## **7.3 Cost vs Performance**

- Higher embedding dimensions and LLM usage improve quality.
  - However, they increase computational and operational cost.
- 

# **8. Testing Strategy**

## 8.1 Testing Approach

The system is validated using:

- Unit testing (individual modules)
- Integration testing (pipeline flow)
- End-to-end query evaluation

## 8.2 Sample Test Queries

- “What is the refund policy?”
- “How do I reset my account password?”
- “Explain billing-related issues in detail.”

## 8.3 Evaluation Metrics

- Retrieval relevance score
  - Response correctness
  - Latency per query
  - HITL activation frequency
- 

## 9. Future Enhancements

The system can be extended in the following directions:

### 9.1 Multi-Document Support

Enable ingestion and querying across multiple knowledge sources.

### 9.2 Feedback Loop Integration

Incorporate user feedback to continuously improve retrieval and responses.

### 9.3 Memory Integration

Add conversational memory for contextual multi-turn interactions.

### 9.4 Deployment Strategy

- Containerized deployment using Docker.
- Cloud hosting (AWS / Azure / GCP).
- Scalable API layer using FastAPI.

Figure 3 – End-to-End RAG Data Flow Pipeline

